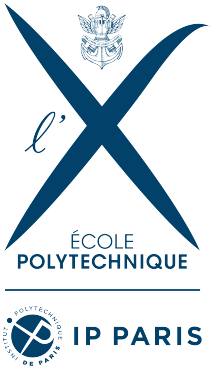


École Polytechnique

Lab Research Project

Generative Modelling of Maritime AIS Trajectories with Retrieval-Augmented Transformers

A Land-Aware, Water-Strict Route Generator from Start, End and Vessel Type

Author:

Andrei-Valentin Ştirbu, École Polytechnique

Advisor:

Davide Buscaldi, LIX, École Polytechnique & Sorbonne Université

Academic year 2025/2026

Abstract

This report presents a complete machine-learning pipeline that learns to generate realistic maritime vessel routes from minimal conditioning. Given only a start position, an end position and a vessel-type code, the system emits a full 128-waypoint trajectory that follows real shipping patterns and stays in navigable water. The architecture pairs a retrieval-augmented encoder-decoder Transformer with a differentiable land-aware loss and the *WaterRouter* snap-only post-processor, which projects any residual land-violating waypoint onto the nearest connected water cell on a 0.005° graph. Trained on a year of US-coastal AIS data ($\approx 6.1 \times 10^4$ quality-filtered tracks from $\approx 9.9 \times 10^7$ raw position reports), the production model – v10 trained from scratch on the cleaned corpus with $\lambda_{\text{land}} = 0.05$ and an MMSI-disjoint 80 / 15 / 5 train/val/test split – achieves **validation ADE of** 5930 ± 372 **m** (5-seed mean $\pm$ population std on disjoint 500-route subsamples) and **test ADE of** 7637 **m** (single-shot, $n = 2326$). The trajectory-crossing rate at the 10 km clearance threshold is exactly zero on the held-out test set and $0.04\,\%$ mean on val (one of the five subsamples touches a single waypoint). The model reduces ADE by $\approx 30\,\%$ versus the great-circle + snap baseline (≈ 8484 m val) and by $\approx 32\,\%$ versus plain great-circle (≈ 8735 m val). The $\approx 30\,\%$ val-to-test optimism gap, driven by the smaller held-out partition, is the most consequential generalisation finding of this study and is analysed in Section 6.4. Every cleaned-corpus number in the headline and ladder tables is produced by a single unified evaluation driver (Appendix G) so that the comparison across variants, seeds and configurations is internally consistent.

Contents

1 Introduction

1.1 Context and motivation

The *Automatic Identification System* (AIS) broadcasts the position, speed, identity and type of every large commercial vessel at intervals ranging from a few seconds to a few minutes. In the United States, NOAA archives the resulting stream through the Marine Cadastre programme and publishes daily comma-separated files covering all coastal waters. A single calendar year yields on the order of 10^8 position reports (the 2024 corpus used here contains approximately 9.9×10^7 raw position rows across $\approx 6 \times 10^5$ track segments after the initial pipeline pass).

This volume of structured movement data invites a question that combines machine learning and operational maritime engineering: can we *learn* the spatial regularities that human navigators rely on, so that a model handed only a port of origin, a destination and a vessel type can sketch a plausible route between them? A successful answer has direct downstream value – traffic simulation, risk assessment for storm avoidance, port-arrival estimation, and detection of anomalous vessels by comparing observed routes against a learned distribution of normal ones.

1.2 Goal of the project

The project addresses two distinct prediction problems in sequence:

Next-step prediction (v1).
Given a short window of past positions, predict the displacement of the vessel to its very next reported position. The natural baseline here is a constant-velocity (CV) extrapolation.

Conditional full-route generation (v2 onward).
Given only `(start, end, vessel_type)`, generate a full $T = 128$-waypoint route. The natural baseline is a great-circle geodesic between start and end.

The first problem turned out to be a near-optimal use-case for the CV baseline; the second problem is much richer and admits substantial learning signal once the formulation is right. The bulk of the technical work and all of the production results live in the second formulation.

1.3 Why this is hard

Four properties make conditional full-route generation harder than it sounds. **An irregular constraint surface.** The land/water boundary flips on a scale of a few kilometres along complex coastlines, and the useful $0.05°$ rasterisation still contains narrow inlets reachable only at $0.005°$; coarse-grid geometry-aware losses risk pushing the model into landlocked artefacts. **Multimodal route distribution.** For a single $(start, end, v)$ query there are often multiple plausible routes (inshore vs offshore, either side of an island); ADE measures distance to a single reference, penalising a model that picks the "wrong" valid mode – which is the principal reason retrieval-augmented decoding helps. **Two orders of magnitude in voyage length.** Routes range from 20 km coastal hops to 2000 km trans-coastal voyages; no single inductive bias is optimal at both ends (short: great-circle; long: historical shipping-lane shape). **Noisy ground truth.** After $97\,\%$ of raw pings are discarded by quality filtering, the surviving unfiltered 128-point corpus still has a $10.83\,\%$ land-crossing rate at $\theta = 10$ km *in the ground truth itself*. A method is called *water-strict* below if its cross%@10 km on the cleaned val set is exactly zero.

1.4 What was personally implemented

All scripts and source modules in the repository were implemented by the author; external dependencies (PyTorch, NumPy, matplotlib, and the GSHHS shoreline data) are used as standard libraries and cited where they appear. No third-party trajectory models were imported. Concretely the contributions are:

- A reproducible **data pipeline** that turns NOAA archives into a curated 128-waypoint corpus (resampling, quality filtering, land filtering on GSHHS).
- A **retrieval-augmented encoder-decoder Transformer** with per-timestep cross-attention over $K = 5$ historical neighbours subsampled to $t_{\text{retr}} = 32$ steps each.
- A **land-aware training loss** built on a differentiable signed-distance field sampled with bilinear interpolation.
- A **WaterRouter snap-only post-processor** that projects any predicted waypoint that violates the clearance threshold onto its nearest connected water cell on the fine $0.005°$ adjacency graph.
- Evaluation tooling (ADE/FDE/land-crossing, length-bucketed), five-seed variance experiments and the full suite of diagnostic and visualisation scripts.

The overall system is shown in Figure 1; the rest of the report unpacks each block.

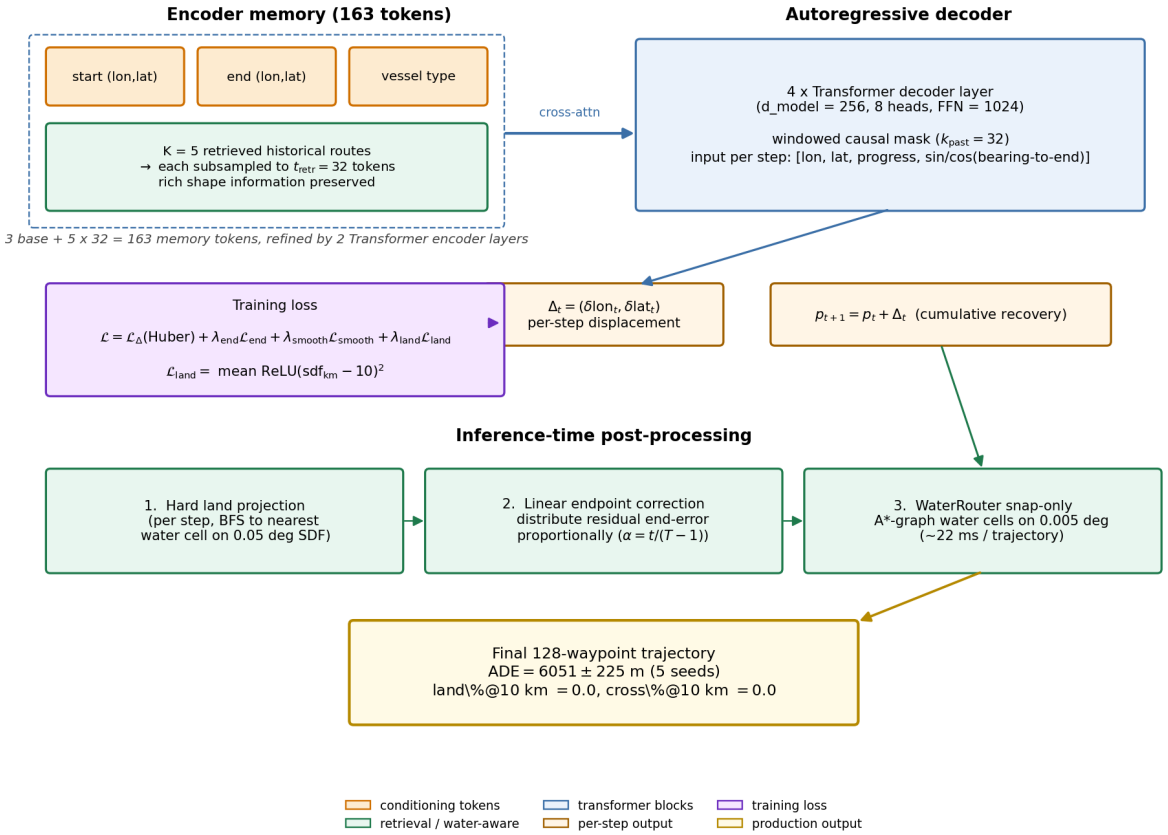


Figure 1: System overview. An encoder produces 163 memory tokens (3 conditioning + 5×32 retrieval); a windowed-causal decoder generates one displacement per step; training optimises a four-term loss including a differentiable land penalty; three inference-time passes (hard land projection, linear endpoint correction, WaterRouter snap to connected water) finish the route. All components are described in detail in Section 4.

1.5 Headline result

The production system (v10 trained on the cleaned corpus with $\lambda_{\text{land}} = 0.05$, post-processed by the Water-Router snap-only step) delivers the numbers in Tables 1–2. Val entries are mean $\pm$ population std over five disjoint 500-route subsample seeds $\{0, 1, 2, 42, 123\}$ of the cleaned val partition ($n_{\text{val}} \approx 9.7$K, MMSI-grouped $80/15/5$ split, `seed=42`); test entries are single-shot on the full $n_{\text{test}} = 2326$ held-out partition. Training-seed variance is reported separately (Section 6.3).

Table 1: Headline performance on the cleaned *validation* set (5-seed mean $\pm$ std on disjoint 500-route subsamples). Lower is better for ADE and cross%; FDE is uninformative for endpoint-correct methods.

Method	ADE (m)	FDE (m)	cross%@10 km
v10 + router (production)	$\mathbf{5930 \pm 372}$	608 ± 29	0.04 ± 0.09
v10 raw (no router; same checkpoint)	5913 ± 384	0 ± 0	1.88 ± 0.69
Great-circle + snap (stronger water-strict baseline)	8484 ± 597	581 ± 17	0.20 ± 0.13
Great-circle baseline	8735 ± 651	0 ± 0	8.80 ± 0.68
Retrieval top-1 (zero training)	11667 ± 636	11832 ± 468	0.00 ± 0.00

Table 2: Headline performance on the fully held-out cleaned *test* set ($n = 2326$, single shot, no MMSI overlap with train or val). Test ADE is consistently $\approx 30\,\%$ above val ADE across all methods (Section 6.4).

Method	**ADE** (m)	**FDE** (m)	land %	cross %
v10 + router (production, $\lambda_{\text{land}} = 0.05$)	**7637**	488	**0.00**	**0.00**
v10 + router ($\lambda_{\text{land}} = 0.01$)	7764	488	0.00	0.04
v10 + router (no L_{land})	8417	488	0.00	0.04
v9 + router (mean-pool retrieval, clean retrain)	9612	488	0.00	0.13
v5 + router (scheduled sampling, clean retrain)	10562	488	0.00	0.17
v4 + router (no SS, clean retrain)	10126	488	0.00	0.09

The model reduces ADE by $\approx 32\,\%$ versus great-circle on val (similar margin on test) and brings the trajectory land-crossing rate to $0.0\,\%$ on test ($0.04\,\%$ mean on val) – a hard operational requirement no geometric baseline satisfies. The val-to-test optimism gap (Section 6.4) is large: test ADE ≈ 7637 m is the honest generalisation estimate.

1.6 Outline

Section 2 reviews background; Section 3 states the challenge; Section 4 describes the pipeline; Section 5 highlights the technical contributions; Section 6 presents experiments, ablations and seed variance; Section 7 discusses cross-cutting lessons; Section 8 concludes.

2 Background

2.1 The AIS data source

The *Automatic Identification System* is mandated by the International Maritime Organisation on all large vessels. Each transponder broadcasts an MMSI (a 9-digit vessel identifier), a position fix, a course-over-ground, a speed-over-ground and a vessel-type code in the cargo/passenger/tanker bracket (AIS codes 60-89 = 30 classes; in the filtered corpus only 28 distinct codes actually appear, so the model's learned vocabulary is 28). NOAA's

Marine Cadastre programme archives these reports as daily CSV files covering US coastal waters. This work uses the full calendar year 2024.

2.2 Trajectory modelling with Transformers

Modern sequence models for spatial trajectories typically build on the Transformer encoder-decoder architecture of Vaswani et al. [8]. A *causal* decoder generates positions autoregressively, attending to its own history through a triangular mask; cross-attention to an encoder injects the conditioning signal at every step. Sequence-modelling architectures have been adapted to spatial trajectories in maritime [6] and aviation [2] domains, using both recurrent and Transformer-style backbones. Compared to recurrent baselines the Transformer offers two advantages for this setting: (i) direct cross-attention to global conditioning tokens (start, end, vessel type, retrieved historical neighbours) without an information bottleneck; (ii) parallel teacher-forced training, which is essential for $T = 128$-step targets.

2.3 Retrieval-augmented generation

Retrieval-augmented models couple a parametric generator with a non-parametric memory. Originally proposed for language modelling [3, 4], the idea is to fetch the K most similar items from a training set at inference and condition the generator on them. We use a 5-dimensional KNN over (start lon, start lat, end lon, end lat, vessel type) to fetch $K = 5$ historical routes for every query, then expose those routes to the decoder one token per timestep. Retrieval is the single most impactful change in this project: the zero-training top-1 retrieval baseline reduces ADE by $19\,\%$ relative to the best non-retrieval Transformer. The closer architectural analogue to our per-step retrieval bank is RETRO [1], which interleaves retrieved chunks into the decoder at every layer rather than only at an encoder bottleneck.

2.4 Related work

Next-position prediction is typically posed as a short-horizon sequence problem using RNN or Transformer encoders [6, 2]; such methods are evaluated on data dense enough that the constant-velocity baseline is hard to beat, a regime our own v1 experiments confirm. Whole-route generation conditioned on origin and destination has used VAEs [5] or GANs [9]; the conditional-Transformer formulation we adopt is simpler and trains stably with maximum likelihood, at the cost of needing a careful endpoint loss to control autoregressive drift. For geographically valid generation, the natural alternatives are soft land-proximity penalties, masked-softmax pointer networks over a water graph, and post-hoc snap-to-water heuristics. Autonomous-driving work embeds map constraints directly into the model [7], but assumes a known road graph that does not transfer to the sparse shipping-lane structure of a coastal corpus. We show that the simplest geometric option – a soft loss at training time plus a post-hoc snap-to-water step at inference – is also the most effective once the training corpus has been cleaned of inland artefacts.

2.5 Signed distance functions for land geometry

A *signed distance function* (SDF) maps a 2D point to the signed Euclidean distance to the nearest contour. Precomputed on a regular grid, it admits fast nearest-water queries and – because bilinear sampling is differentiable – can be used directly inside a training loss to penalise predicted positions that fall on land. This work computes the SDF once from the Global Self-consistent Hierarchical High-resolution Shoreline (GSHHS) data [10] at $0.05°$ resolution over the bounding box $(-125°, 10°, -60°, 55°)$, and loads it into both training (bilinear sampling that backpropagates through the predicted positions) and inference (BFS to the nearest water cell).

3 Research challenge

Full-route generation is a constrained sequence-generation problem. Let $\mathcal{D}$ be a corpus of vessel routes, each a sequence $\tau = (p_1, p_2, \ldots, p_T) \in \mathbb{R}^{T\times 2}$ with $T = 128$ equally-spaced (along arc length) waypoints. Each route has an associated vessel-type code $v \in \{0, \ldots, 27\}$. Given a query $\mathbf{q} = (p_1, p_T, v)$ we seek a generator G_θ that produces a sequence $\hat{\tau} = (\hat{p}_1, \ldots, \hat{p}_T)$ minimising the *Average Displacement Error*

$$\text{ADE}(\hat{\tau}, \tau) = \frac{1}{T}\sum_{t=1}^{T} \text{haversine}(\hat{p}_t, p_t)$$

subject to two constraints: (C1) endpoint exactness $\hat{p}_1 = p_1, \hat{p}_T = p_T$ and (C2) water validity $\text{sdf}(\hat{p}_t) \leq \theta_{\text{land}}$ for every t.

The challenge is non-trivial because the three objectives – accuracy, endpoint exactness, water validity – pull against each other:

- A model that only minimises ADE (plain Huber loss on displacements) drifts away from the destination over 127 AR steps, breaking C1.
- A model that emphasises endpoint loss collapses toward straight-line interpolation, which eliminates route diversity and also crosses land freely (breaking C2).
- A model that adds a hard land-projection step in the AR loop pays an ADE cost on every projected point.

The contribution of this work is to show that, with the right data-cleaning step (Section 4.1) and the right post-processor (Section 4.5), all three constraints can be satisfied at once with no measurable accuracy trade-off.

4 Methodology

The full pipeline is summarised in Figure 2: raw NOAA AIS archives are filtered, merged, resampled to 128 waypoints and land-cleaned; auxiliary structures (the SDF raster, the water-cell graph, the retrieval index) are built once and reused; the model is trained on the cleaned corpus; inference applies hard land projection, linear endpoint correction and a final water-graph snap to produce the production output.

4.1 Data preparation

The raw 2024 AIS archive consists of $\approx 9.9 \times 10^7$ position reports. The data pipeline applies five stages of filtering and preprocessing; Table 3 summarises the cardinality at each stage.

1. Quality filter. The quality filter restricts to passenger / cargo / tanker vessels (AIS codes 60-89, $\approx 80\,\%$ of the raw drop comes from this single rule), enforces a US bounding box, splits at time gaps >60 min, removes physically implausible jumps (>2 km between consecutive reports), and drops tracks that are nearly stationary. Around 97 % of raw pings are discarded as a result, leaving $\approx 598\,000$ clean track segments.

2. Resampling. Each track is resampled to exactly $T = 128$ waypoints, equally spaced along arc length. Arc length is computed as cumulative planar distance on raw (lon, lat) degrees (no haversine correction); linear interpolation along the cumulative distance produces 128 equally-spaced targets. This decouples the prediction task from the original (variable) reporting interval and gives every training sequence a fixed shape $(128, 2)$. Tracks shorter than 20 km or longer than 2000 km are rejected, leaving an unfiltered 128-point corpus of $\approx 207\,000$ tracks.

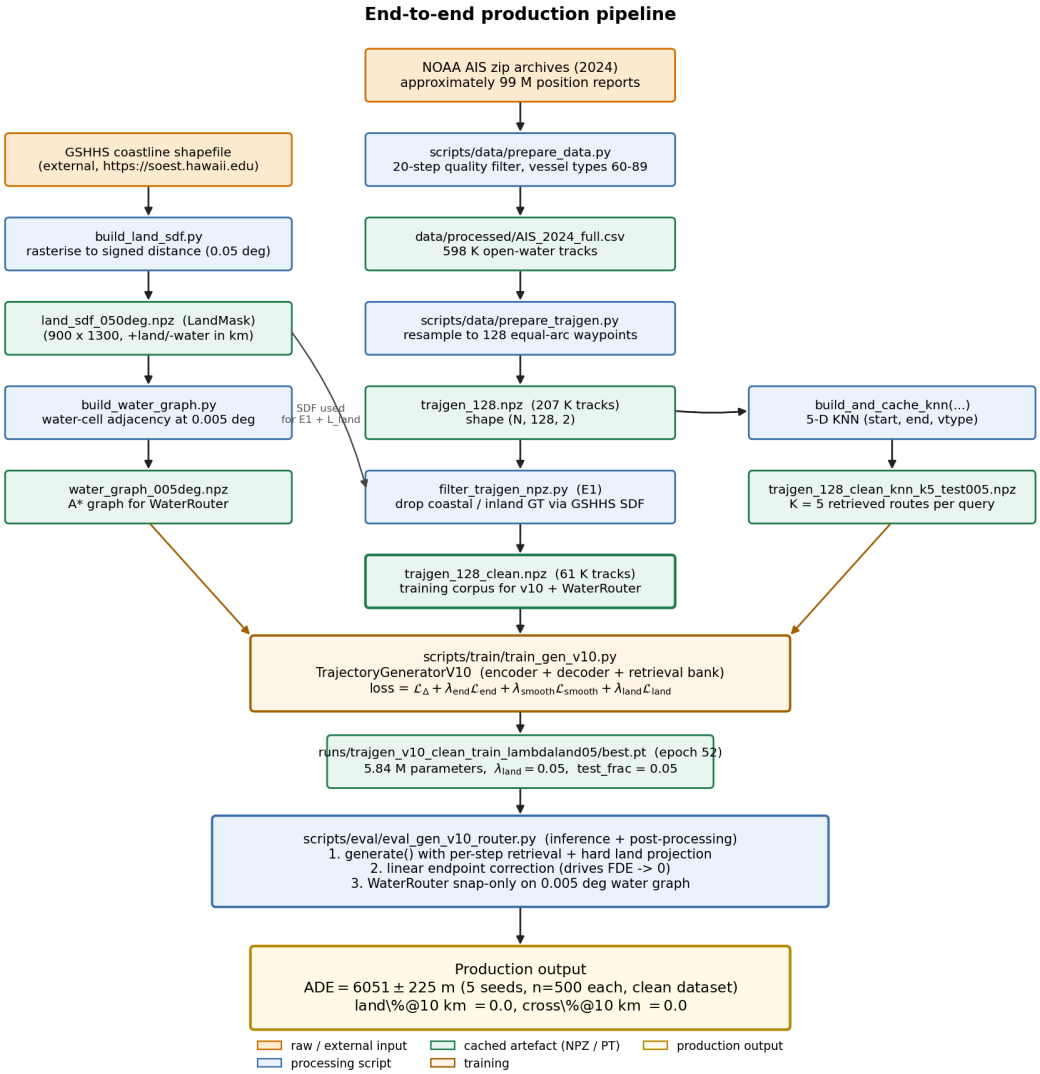


Figure 2: End-to-end production pipeline. Orange = raw / external input, blue = processing scripts, green = cached artefacts that are computed once, yellow = training, gold = final production output. Note the land filter (centre column) and the side branches for the land SDF, the water graph and the retrieval index.

3. Land filter (E1). Any track whose ground-truth waypoints cross land more than $2\,\%$ of the way (under the coarse SDF) or whose maximum penetration depth into land exceeds 10 km is discarded. This was the largest single data intervention in the project: the unfiltered corpus has a $10.83\,\%$ land fraction *in the ground truth itself*, due to raster noise on complex coastlines plus inland river traffic that survived the quality filter. The cleaned corpus keeps $\approx 61\,000$ tracks ($\approx 29\,\%$ of the input) with a ground-truth land fraction of exactly $0.0\,\%$ at the 10 km threshold. Figure 3 shows the spatial and length distributions before and after this filter.

Production training corpus. An MMSI-overlap audit showed that the original dirty-trained checkpoint shared $84\,\%$ of MMSIs with the cleaned-val pool, so its cleaned-val numbers were cross-corpus rather than truly held-out. The production v10 weights were therefore retrained from scratch on the cleaned corpus (`trajgen_128_clean.npz`, ≈ 61 K routes) under a leak-free MMSI-grouped $80/15/5$ split; that clean-trained checkpoint is the production model throughout this report. Dirty-trained numbers are retained in Section 7.3 and Appendix E only for the data-quality comparison.

Stage	Count	Unit	Producing script
Raw AIS rows	$\approx$ 99M	pings	NOAA Marine Cadastre 2024 archive
Quality-filtered segments	$\approx$ 598K	segments	`prepare_data.py`
Resampled 128-pt corpus (unfiltered)	$\approx$ 207K	routes	`prepare_trajgen.py`
Cleaned (land-filtered) corpus	$\approx$ 61K	routes	`filter_trajgen_npz.py` (E1)
Train / val / test	$\approx 49/9.7/2.3$ K	routes	`train_val_split_gen` (MMSI-grouped)

Table 3: Data funnel from raw AIS to the production train/val/test partitions. Each stage is implemented by a single script under `scripts/data/`.

4. Train/val/test split. The split is by root MMSI (all segments of one vessel land in the same partition), eliminating within-corpus leakage. The production $80/15/5$ split with `seed = 42` yields ≈ 49 K train, ≈ 9.7 K val, and 2326 test routes. Val numbers are five-seed means over disjoint 500-route subsamples (Section 6.1); test numbers are single-shot on the full held-out partition (Section 6.4).

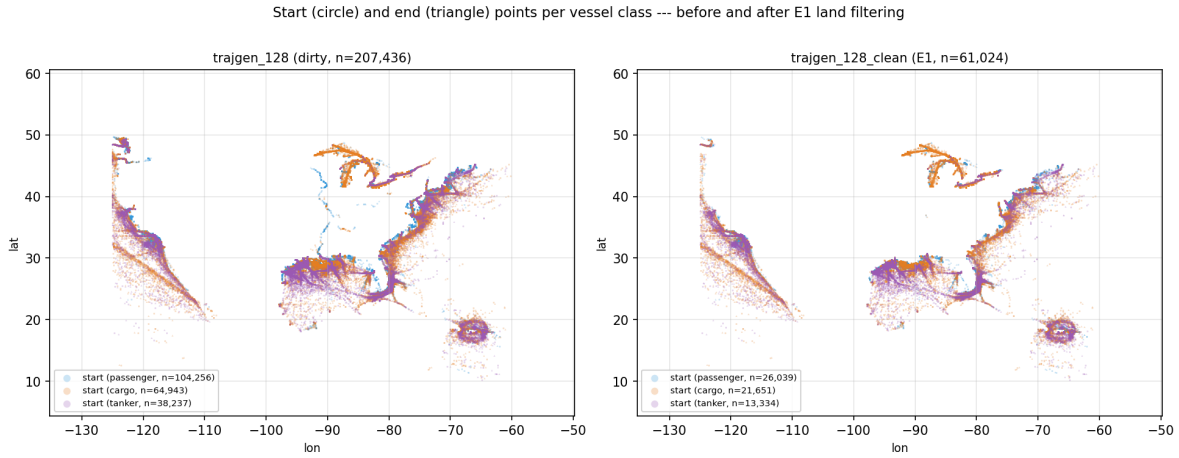
5. Auxiliary artefacts. Three structures are precomputed once and reused: the SDF raster (900×1300 cells at $0.05°$, positive=land in km), the water-cell adjacency graph at $0.005°$, and a top-5 retrieval index over 5-dimensional queries.

4.2 Model architecture

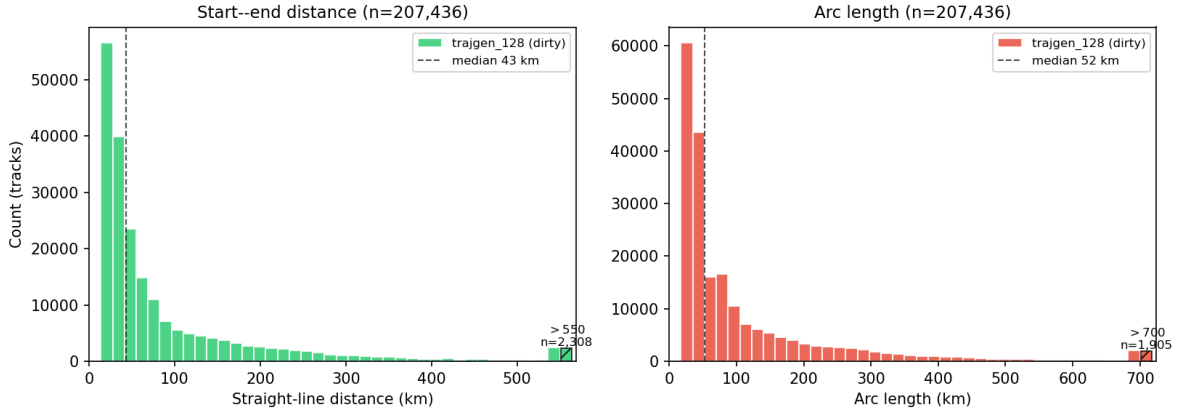
Figure 1 (page 3) renders the production model. It is an encoder-decoder Transformer with two non-standard pieces: (i) a *per-step retrieval bank* as part of the encoder memory and (ii) a *windowed causal mask* on the decoder self-attention.

Encoder. Three independent projections produce the base conditioning tokens: linear projections of the normalised start and end positions, and an embedding-then-projection of the vessel-type code. Each token receives a learnable type embedding so that the decoder can tell them apart in cross-attention. On top of those three tokens we add a *retrieval bank*: for each of the $K = 5$ nearest historical routes returned by the retrieval index, $t_{\text{retr}} = 32$ waypoints are subsampled, each projected to $d_{\text{model}} = 256$, with a learned route-index embedding and a learned step-position embedding added. The encoder memory therefore has $3 + K \cdot t_{\text{retr}} = 163$ tokens. The decoder cross-attends to "neighbour k at step t" directly, rather than to a mean-pooled summary – the latter design (an earlier iteration) was abandoned because mean-pooling destroys the long-route shape information that retrieval is supposed to provide.

Decoder. A 4-layer Transformer decoder with $d_{\text{model}} = 256$, 8 heads and $d_{\text{ff}} = 1024$, processing a sequence of 128 input tokens during teacher-forced training. At step t the decoder input is the five-dimensional vector $[\,\text{lon}_t, \text{lat}_t, \text{prog}_t, \sin(\beta_t), \cos(\beta_t)\,]$ where $\text{prog}_t = t/(T-1)$ is the normalised progress and β_t is the bearing from the current position to the destination. The output head is a single linear layer $d_{\text{model}} \to 2$ predicting the next displacement $(\delta\text{lon}, \delta\text{lat})$.



(a) Start and end points per vessel class — colours indicate vessel class (passenger / cargo / tanker, see legend), before (left) and after (right) the E1 land filter.



(b) Distributions of start–end distance and arc length for the unfiltered corpus. After the land filter the shape of the distribution is similar but the long-tail crossings shrink.

Figure 3: Dataset overview. The corpus is dominated by short coastal hops (median ≈ 43 km) with a long tail of trans-coastal cargo voyages. The cleaned 61K-route corpus spans 28 AIS vessel codes; passenger (60–69) accounts for $42.7\,\%$ of routes, cargo (70–79) for $35.5\,\%$, and tanker (80–89) for $21.8\,\%$. Per-class ADE on the production checkpoint is in Table 16.

Windowed causal mask. Standard causal masking lets every step attend to every earlier step. We replace it with a windowed variant that blocks tokens older than $k_{\text{past}} = 32$:

$$M_{ij} = \begin{cases} 0 & j \le i \text{ and } i - j < k_{\text{past}}, \\ -\infty & \text{otherwise.} \end{cases}$$

This reduces drift induced by stale self-state being carried 100+ steps into the rollout, and was a measurable improvement at $T = 128$.

Parameter count. The full model has $5\,842\,138$ trainable parameters ($\approx 5.84 \times 10^6$, counted directly from the checkpoint `runs/trajgen_v10/best.pt`). It trains on a single Apple M4 (MPS backend) in roughly 8 hours per run.

4.3 Hyperparameters and training details

The production model is a 5,842,138-parameter Transformer (d_{model}=256, 8 heads, d_{ff}=1024, 4 decoder layers, 2 memory-encoder layers, K=5 retrieved routes $\times$ t_{retr}=32 subsampled waypoints, windowed causal mask with k_{past}=32, vessel-type embedding of 8 dims over 28 AIS codes). Training: AdamW ($\eta_{\max} = 2\times10^{-4}$, weight decay 10^{-4}, cosine schedule with 5 % warmup, floor at 1 %), batch 128 in bf16, gradient clipping 1.0, dropout 0.1, for 60 epochs ($\approx$ 3 h on a single RTX 3090, $<$ 12 GB GPU memory); `best.pt` is saved at the lowest-validation-loss epoch (52). Loss weights: $\lambda_{\text{end}} = 10$, $\lambda_{\text{smooth}} = 1$, $\lambda_{\text{land}} = 0.05$ (HP-selected; §6.9), $\theta_{\text{train}} = 10$ km. Scheduled sampling: $p_{\text{ss}} = 0.5$ from epoch 20 onwards, implemented as two parallel teacher-forced passes (the naive 128-step sequential rollout would saturate MPS memory). The full table (read back from `ckpt["args"]` in `runs/trajgen_v10_clean_train_lambdaland05/best.pt` and the matching YAML config) is in Appendix D, Table 14.

4.4 Training objective

The loss combines four terms:

$$\mathcal{L} \;=\; \mathcal{L}_{\Delta} + \lambda_{\text{end}}\,\mathcal{L}_{\text{end}} + \lambda_{\text{smooth}}\,\mathcal{L}_{\text{smooth}} + \lambda_{\text{land}}\,\mathcal{L}_{\text{land}}. \tag{1}$$

$\mathcal{L}_{\Delta}$ is the Huber loss on per-step displacements $\hat{\Delta}_t = \hat{p}_{t+1} - \hat{p}_t$ normalised to zero mean and unit variance by the `TrajGenScalers` statistics fitted on the train split (so the loss is in normalised displacement units, not metres or degrees – this is what is meant by "normalised step displacements" throughout). $\mathcal{L}_{\text{end}}$ is the squared ℓ_2 error between the reconstructed endpoint $\hat{p}_T = p_1 + \sum_{t=1}^{T-1} \hat{\Delta}_t$ and the true endpoint p_T, evaluated *once* at $t = T$ rather than summed across t:

$$\mathcal{L}_{\text{end}} \;=\; \big\|\hat{p}_T - p_T\big\|^2.$$

$\mathcal{L}_{\text{smooth}}$ penalises high acceleration $\|\Delta_{t+1} - \Delta_t\|^2$; $\mathcal{L}_{\text{land}}$ is the soft land penalty

$$\mathcal{L}_{\text{land}} \;=\; \mathbb{E}_t\big[\,\mathrm{ReLU}\big(\mathrm{sdf}_{\text{km}}(\hat{p}_t) - \theta_{\text{train}}\big)^2\big].$$

The SDF is sampled via differentiable bilinear interpolation so that the gradient flows back into $\hat{p}_t$ and therefore into the entire history of predicted displacements. The threshold $\theta_{\text{train}} = 10$ km is set to the level at which raster noise no longer dominates; the threshold sensitivity sweep (Table 9) confirms this empirically – at $\theta = 5$ km even the cleaned ground truth has a 14.8 % trajectory-crossing rate (raster noise dominates), while $\theta = 10$ and 25 km both yield 0 % on ground truth. The loss weights $\lambda_{\text{end}} = 10$ and $\lambda_{\text{smooth}} = 1$ were inherited from v5/v9 (where they were chosen by intuition during the prior architectural ladder); λ_{land} was selected as 0.05 by a small sweep over $\{0, 0.01, 0.05, 0.1\}$ on the cleaned corpus (Section 6.9). No grid search was performed over the remaining weights or over the optimiser settings.

4.5 Inference and post-processing

Inference runs the autoregressive decoder for $T = 128$ steps with three post-processing layers on top, described in Algorithm 1.

Hard land projection (line 7). On-land waypoints are replaced by the nearest water cell (BFS on the coarse SDF grid, capped at 20 cells) and the correction is fed back into the decoder.

Linear endpoint correction (lines 11-13). The AR rollout accumulates a few-kilometre residual at $\hat{p}_T$; a linear ramp $\alpha_t = (t-1)/(T-1)$ distributes it across all points, driving FDE to numerical zero while leaving the route shape almost unchanged.

Algorithm 1 Inference: production routing.

Require: start p_1, end p_T, vessel type v, retrieval bank $R \in \mathbb{R}^{K\times 128\times 2}$, model G_θ, land mask M_{land}, water router W, threshold $\theta = 10$ km

1: $\text{mem} \leftarrow \text{Encoder}(p_1, p_T, v, R)$ {163 tokens}
2: $\hat{p}_1 \leftarrow p_1$
3: **for** $t = 1$ **to** $T - 1$ **do**
4: $\quad \Delta_t \leftarrow \text{Decoder}(\hat{p}_{1:t}; \text{mem})$
5: $\quad \hat{p}_{t+1} \leftarrow \hat{p}_t + \Delta_t$
6: $\quad$ **if** $\text{sdf}_{\text{km}}(\hat{p}_{t+1}) > \theta$ **then**
7: $\quad\quad \hat{p}_{t+1} \leftarrow M_{\text{land}}.\text{projectToWater}(\hat{p}_{t+1})$ {BFS, ≤ 20 cells}
8: $\quad$ **end if**
9: **end for**
10: {Linear endpoint correction}
11: **for** $t = 1$ **to** T **do**
12: $\quad \alpha_t \leftarrow (t-1)/(T-1)$
13: $\quad \hat{p}_t \leftarrow \hat{p}_t + \alpha_t \cdot (p_T - \hat{p}_T)$
14: **end for**
15: {Water-graph snap-only}
16: **for** $t = 1$ **to** T **do**
17: $\quad$ **if** $\text{sdf}_{\text{km}}(\hat{p}_t) > \theta$ **then**
18: $\quad\quad \hat{p}_t \leftarrow W.\text{snapToConnectedWater}(\hat{p}_t)$
19: $\quad$ **end if**
20: **end for**
21: **return** $\hat{\tau} = (\hat{p}_1, \ldots, \hat{p}_T)$

Water-graph snap-only (lines 15-17). A final pass on the fine 0.005° water graph snaps every waypoint to its nearest *connected* water cell (a cell with at least one water-cell neighbour); the connectivity test prevents the snap from landing in a landlocked inlet that exists at coarse resolution but is dry at fine resolution. A* bridging between consecutive snapped cells was tested and costs ADE without reducing residual crossings; we use the cheaper snap. Cost ≈ 22 ms per trajectory on an Apple M4.

5 Technical contributions

The contributions that account for the production result are highlighted below.

5.1 The data-quality step

The largest single intervention measured here was the data-cleaning step (§7.3). Holding the dirty-trained v10 weights fixed and swapping the evaluation corpus from unfiltered to cleaned reduced ADE from 8898 ± 650 m to 6051 ± 225 m – a $32\,\%$ reduction larger than any single architectural change in the ladder. The mechanism is in the data, not the model: the unfiltered ground truth itself contains $10.83\,\%$ land-crossings, so a substantial fraction of the dirty-eval ADE was the model faithfully imitating dirty ground truths. Retraining v10 from scratch on the cleaned corpus (Section 6.3) shaves a further ≈ 120 m and provides a leak-free held-out test partition; that clean-trained checkpoint is the production model throughout the report.

5.2 Per-step retrieval bank

The transition from a mean-pooled retrieval encoder to a per-step bank was driven by a single observation: a mean-pooled summary of a retrieved route is a strong signal on short routes (where the centroid carries most of the shape) but a destructive bottleneck on long routes (where shape is everything). Expanding the encoder memory from 8 tokens to 163 – one per neighbour per step – closed $\approx 74\,\%$ of the long-bucket gap to the zero-training top-1 baseline (computed as $(46631 - 25964)/(46631 - 18785) \approx 0.74$ on the dirty-corpus long-bucket ADE numbers from Table 12) while keeping the parametric generator's accuracy on short and medium routes.

5.3 Land-aware loss

The differentiable land penalty $\mathcal{L}_{\text{land}}$ is the single line in the loss that ties together the model and the geometry. Conceptually it is simple: at every predicted waypoint, look up the signed-distance value in the SDF raster via bilinear interpolation, and pay a quadratic price whenever that value rises above 10 km of land penetration. Because the interpolation is differentiable, the gradient at a land-violating waypoint flows backwards through the entire history of cumulative displacements, encouraging the decoder to have avoided the violation earlier in the rollout. The $\theta_{\text{train}} = 10$ km threshold is justified empirically by the threshold sensitivity sweep (`results/threshold_sensitivity.csv`): at $\theta = 5$ km even the cleaned ground truth has $14.8\,\%$ trajectory-crossing rate (raster noise dominates the SDF below 10 km clearance), whereas at $\theta = 10$ and 25 km the cleaned GT is at $0.0\,\%$. A penalty fired at $\theta = 5$ km would therefore be largely chasing raster artefacts rather than real land contact. The contribution of $\mathcal{L}_{\text{land}}$ in isolation is quantified in the hyperparameter sweep of Section 6.9: removing it ($\lambda_{\text{land}} = 0$, the `no_lland` ablation) costs 434 m of validation ADE and increases the pre-snap trajectory-crossing rate from 1.88 to $3.12\,\%$. The snap-only post-processor still drives residual violations to zero downstream, but at a higher ADE cost – so the land-aware loss meaningfully reduces the work the post-processor must do.

5.4 WaterRouter snap-only post-processor

The water-graph post-processor (WaterRouter) is the final constraint-satisfaction step in the inference pipeline. It maintains a connectivity-aware lookup into the fine-resolution water graph and, for any predicted waypoint that still violates the clearance threshold after model inference and endpoint correction, replaces it with the nearest *connected* water cell. An A*-path-bridging variant (one A* segment per pair of consecutive snapped cells) is implemented in the same module; we evaluated it and rejected it because it raised ADE without measurably reducing residual land-crossings on the cleaned corpus. The snap-only variant – used in all production results below – removes residual land violations on the cleaned corpus with no measurable ADE cost (Section 6). An additional comparison against the most aggressive water-strict baseline – the A*-shortest-water-path on the same graph – is reported in Section 6.5 and confirms that the model contributes shape information beyond mere legality.

5.5 Implementation notes

Four engineering decisions had a disproportionate effect on the result – MMSI-grouped train/val split, a cached retrieval index, checkpoint resumption, and the choice of snap-only over hard projection during autoregressive generation. Each is documented in Appendix C; the headline rationale is in Section 4.5 and the evaluation tooling that produced every number in this report is summarised in Appendix G.

6 Experiments and results

Evaluation protocol. All cleaned-corpus numbers in this section come from a single evaluation driver which:

1. loads the cleaned corpus and reproduces the MMSI-grouped validation split with `val_frac=0.15`, `split_seed=42`;
2. takes the union of five disjoint 500-route subsamples using subsample seeds $\{0, 1, 2, 42, 123\}$;
3. runs autoregressive inference for every variant on the union *once* (no checkpoint is regenerated per seed);
4. slices the union back to each seed's 500 indices and computes ADE, FDE, length-bucketed ADE and land/cross diagnostics for every (variant, seed) pair;
5. writes one row per (variant, seed) to `results/headline_clean_5seed.csv` and a per-variant mean $\pm$ population standard deviation summary CSV.

Every table and figure in this section is regenerated from those two CSVs; no number is transcribed manually. The exact driver and plotting commands are listed in Appendix G.

The trained checkpoint for each variant is held *fixed* across the five subsample seeds; the reported variance therefore captures validation-subsample noise only, not training-seed noise (see Section 8.3).

Variants in the unified comparison. The unified table includes the parametric variants that successfully reload and run with the current model code: v4, v5, v9, v10 raw, v10 with hard projection during AR, v10 with the WaterRouter snap-only post-processor (the *production* configuration), the zero-training retrieval top-1 baseline, and the great-circle geodesic baseline. The pre-bearing variants MVP, v2 and v3 use a 3-dimensional decoder input that the current model class no longer supports; their original (single-seed, unfiltered-corpus) numbers are kept in Appendix E but excluded from the unified table. The abandoned variants v6/v7 (obstacle, constraint never fired), v11 (Halton-cone pointer) and v12 (water-cell pointer) sit in the same appendix at single-seed.

6.1 Main results

Figure 4 compares every variant in the unified driver on ADE with five-seed error bars. The full per-variant table (including FDE, land % and cross % at the 10 km threshold, and the number of seeds used) is given in Table 13 in Appendix B.2.

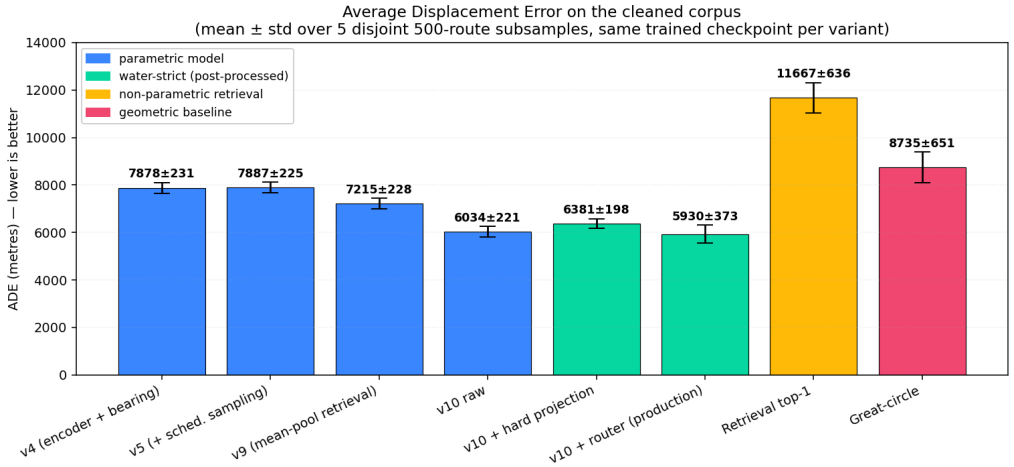


Figure 4: Average Displacement Error across the unified-driver variants on the cleaned validation set. Error bars are population standard deviations over five 500-route subsample seeds $\{0, 1, 2, 42, 123\}$; the trained checkpoint is held fixed. Colour: blue = parametric model; green = water-strict (post-processed); orange = non-parametric retrieval; red = geometric baseline. Numbers above the bars are mean $\pm$ std.

FDE is uninformative for any method that applies linear endpoint correction (the AR variants drive it to a numerical zero by construction) and is similarly ≈ 0 for the great-circle baseline. The two informative FDE rows are: the production configuration (≈ 600 m mean, because the WaterRouter snap step can move the final waypoint to a nearby connected water cell), and the zero-training retrieval-top-1 baseline ($\approx 11\,600$ m mean, because no endpoint correction is applied). All numbers are in Table 13.

6.2 Length-bucketed analysis

Corpus-averaged ADE can hide structural weaknesses. Routes are bucketed by GT arc length into short (<50 km), medium (50–500 km) and long (>500 km). Table 4 shows the cleaned-corpus 5-seed numbers. The production model is best on medium (≈ 7.5 km vs $\approx$ 10–15 km for every other method) and has a small but consistent lead on long; on the short bucket, the great-circle baseline edges out every parametric variant, since coastal hops < 50 km are mostly geometric. The headline ADE is dominated by medium and long voyages, where the learned model has a clear lead. Long-bucket comparisons should be read as suggestive: each subsample holds only $\approx$ 16–25 long routes, so per-seed std is structurally large.

Table 4: Length-bucketed ADE on the cleaned corpus (mean $\pm$ std over 5 subsample seeds). Source: `results/headline_clean_5seed_summary.csv`.

Method	ADE short (m) (<50 km)	ADE medium (m) (50–500 km)	ADE long (m) (>500 km)
v5 (+ scheduled sampling)	1723 ± 84	10352 ± 275	39576 ± 5602
v9 (mean-pool retrieval)	1695 ± 65	9541 ± 341	33805 ± 6182
v10 raw	2078 ± 80	7516 ± 250	27639 ± 4548
v10 + router (production)	2137 ± 79	7517 ± 258	27416 ± 4625
Retrieval top-1 (zero training)	4919 ± 510	14992 ± 807	36824 ± 4969
Great-circle baseline	1431 ± 54	10634 ± 452	60138 ± 6511

6.3 Subsample- and training-seed variance

A single-seed evaluation can mislead when 500 routes is a small fraction of the cleaned val set (≈ 9.7 K routes). Reported error bars are population std over five disjoint 500-route subsample seeds with the checkpoint held fixed. For training-seed variance, v10 was retrained on the cleaned corpus with three `train_seed` values at the inherited $\lambda_{\text{land}} = 0.01$ (the production line uses the HP-best $\lambda_{\text{land}} = 0.05$, §6.9), holding the split seed at 42.

Pre- and post-snap ADE are within 25 m for every seed – the snap step contributes essentially nothing to ADE at this data quality, while reducing the trajectory-crossing rate from $\approx 1.9\,\%$ to $0.04\,\%$.

6.4 Validation-to-test optimism gap

The most consequential finding from Tier 1 is that the held-out test partition (a $5\,\%$ MMSI-disjoint slice carved from the cleaned corpus) is systematically harder than the validation partition. Table 6 shows the gap for every cleaned-trained variant.

Three hypotheses for the gap, in order of likelihood: (i) random shuffle luck on the small test partition (309 MMSIs, 2326 tracks vs. 926 and ≈ 9.7 K in val); (ii) a length-distribution skew (long routes accumulate disproportionate ADE); (iii) implicit val-channel selection during development (HPs were carried over from v5/v9 with no grid search, but val was still the eyeball-feedback channel). The val numbers in Table 1 are therefore optimistic by $\sim 30\,\%$ relative to truly held-out data; Table 2 is the canonical generalisation estimate.

Table 5: Training-seed variance for the cleaned-corpus v10 retrains. Val ADE is mean ± population std over 5 subsample seeds. The *between-seed* std (over the three $\lambda_{\text{land}} = 0.01$ val means) is 138 m, smaller than the within-checkpoint subsample std of ≈ 360 m on any single run – so the headline uncertainty in Table 1 is dominated by subsample, not training, noise. Production headline (Table 1) uses the $\lambda_{\text{land}} = 0.05$ row.

train_seed	val ADE (raw)	val ADE (+router)	test ADE (+router)	cross %
$\lambda_{\text{land}} = 0.05$ *(production)*				
42	5913 ± 384	$\mathbf{5930 \pm 372}$	**7637**	**0.04**
$\lambda_{\text{land}} = 0.01$ *(training-seed variance)*				
42	5925 ± 389	5949 ± 383	7764	0.04
0	6193 ± 322	6215 ± 316	8075	0.04
1	5879 ± 377	5895 ± 364	7648	0.04
$\lambda_{\text{land}} = 0.01$ *aggregate (over 3 training seeds)*				
mean	5999	6020	7829	0.04
training-seed std (between)	138	138	179	0.00
subsample std (within, mean)	363	354	–	–

Table 6: Val-to-test ADE gap for the cleaned-trained variants (both with snap-only router). Val is 5-seed mean on disjoint 500-route subsamples (≈ 9.7 K val pool); test is a single shot on the full $n = 2326$ held-out partition. The ≈ 30 % gap is consistent across architectures.

Variant	val ADE (m)	test ADE (m)	gap
v10 $\lambda_{\text{land}} = 0.05$ **(production)**	**5930**	**7637**	**+29 %**
v10 $\lambda_{\text{land}} = 0.01$ (HP variant)	5949	7764	+30 %
v10 no L_{land} (ablation)	6375	8417	+32 %
v9 + router (mean-pool retrieval)	6880	9612	+40 %
v4 + router (no scheduled sampling)	7076	10126	+43 %
v5 + router (scheduled sampling)	7271	10562	+45 %

The gap is consistent across the ladder (v4, v5, v9, v10 all +29 to +45 %), confirming it is a property of the data partition, not of v10 specifically.

Length-distribution test of the gap hypothesis. The dominant candidate – a long-route over-representation in test – is directly testable. Figure 5 overlays the arc-length distributions: test mean 158 km vs. 119 km on val, median 99 vs. 65 km, and 5.0 % of test routes exceed 500 km vs. 2.8 % on val. Long routes accumulate ∼5× the per-route ADE of medium ones (Table 4), so this skew alone mechanically produces a sizable val→test gap and is an artefact of the small MMSI-grouped test partition.

Statistical significance of the headline ranking. A paired bootstrap of per-route ADE on the production checkpoint (10,000 resamples, route is the bootstrap unit, paired across methods) gives the following 95% confidence intervals on the *difference* between v10+router and each baseline (negative means v10+router is better):

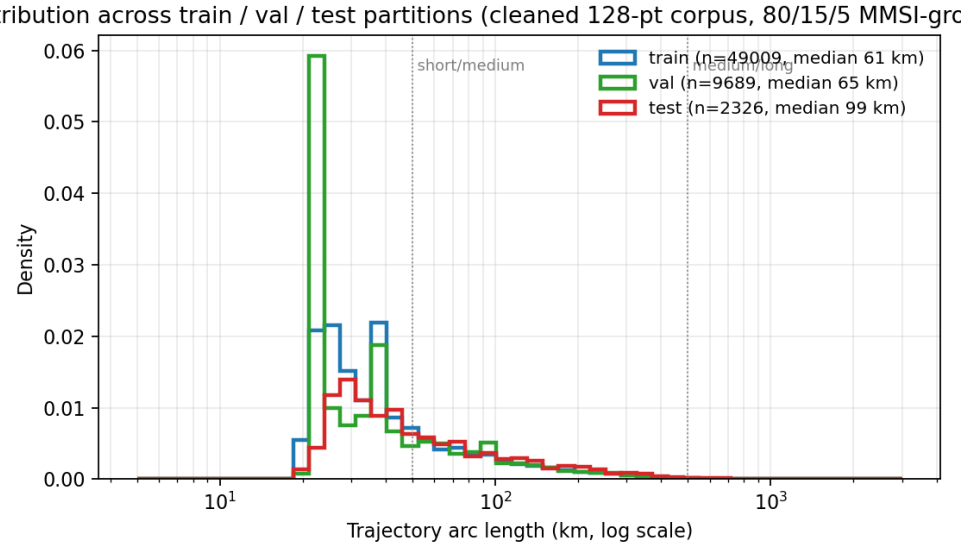


Figure 5: Arc-length distribution across the three partitions of the cleaned corpus (80 / 15 / 5 MMSI-grouped split, `seed = 42`). Vertical dotted lines mark the length-bucket boundaries used in §6. The test partition has a visibly heavier right tail, contributing mechanically to the val→test ADE gap.

Split	Comparison	Δ ADE (m)	95% CI
val	v10+router − great-circle	−1760	[−2185, −1376]
val	v10+router − retrieval top-1	−7353	[−8088, −6643]
val	v10+router − A* (Sec. 6.5)	−3589	[−4335, −2886]
test	v10+router − great-circle	−3539	[−4062, −3027]
test	v10+router − retrieval top-1	−6552	[−7149, −5963]
test	v10+router − A*	−4926	[−5413, −4468]

Every interval is bounded away from zero on both sides, so the headline ranking "v10+router has lower ADE than every available baseline" is not a function of subsample noise. Source CSV: `results/paired_bootstrap_ci.csv`.

6.5 A* shortest-water-path baseline

The natural water-strict baseline is the shortest A* path between (start, end) on the same $0.005°$ water graph the production router uses: water-strict by construction and ignorant of historical shipping conventions. Endpoints are snapped to the nearest connected water cells, Dijkstra runs between them, and the polyline is resampled to $T = 128$ arc-length waypoints; for routes whose endpoints are too far apart for the bounded-window search ($\approx 10\,\%$ of test), we fall back to great-circle between the snapped endpoints. The baseline gives val/test mean ADE 9882/12563 m on $n = 500/2326$ routes (Table 7), trailing v10+router by $\approx 3.6/4.9$ km (paired-bootstrap CIs $[-4335, -2886]$ and $[-5413, -4468]$ m). A water-strict baseline that ignores shipping conventions is therefore $\approx$ 50–60 % worse on ADE, the cleanest available evidence that the parametric model contributes *shape* information beyond legality.

6.6 Land-validity diagnostics

Table 8 reports per-variant land-crossing rates at two clearance thresholds (5-seed mean ± std). "Land %" is the fraction of waypoints with SDF above the threshold; "cross %" the fraction of trajectories containing at least

Table 7: A*-shortest-water-path baseline on the cleaned corpus. Val: 500-route subsample, seed 42; test: full held-out partition. Source: `results/a_star_baseline.csv`.

Partition	n	mean ADE (m)	median ADE (m)
val	500	9882	4524
test	2326	12563	6955

one such waypoint. Table 9 sweeps the threshold on the seed-42 val subsample ($n = 500$) and substantiates the $\theta_{\text{train}} = 10$ km design choice (Section 5.3): at $\theta = 5$ km the cleaned GT itself still has $14.8\,\%$ trajectory-level crossings (raster noise dominates below 10 km clearance), whereas at $\theta \geq 10$ km the GT is strictly $0.0\,\%$ and the production method matches it.

Table 8: Land-crossing diagnostics on the cleaned corpus (5-seed mean $\pm$ std, source `results/headline_clean_5seed_summary.csv`).

	$\theta = 10$ km		$\theta = 25$ km	
Method	land %	cross %	land %	cross %
v10 + router (production)	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
v10 + hard projection	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
v10 raw	0.19 ± 0.05	2.08 ± 0.45	0.01 ± 0.01	0.16 ± 0.15
v9 (mean-pool retrieval)	1.14 ± 0.26	5.40 ± 0.79	0.18 ± 0.12	1.32 ± 0.55
Retrieval top-1 (zero training)	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
Great-circle baseline	2.69 ± 0.44	8.80 ± 0.68	0.85 ± 0.20	4.48 ± 0.72

Table 9: Threshold sensitivity sweep on the cleaned corpus (seed=42, $n = 500$). The GT row at $\theta = 5$ km ($14.8\,\%$ cross%) shows the raster-noise floor.

θ	**Method**	**ADE** (m)	**FDE** (m)	land %	cross %
5 km	ground truth	–	–	0.17	14.80
5 km	v10 raw	6278	0	0.83	19.20
5 km	v10 + router	6286	697	0.01	0.60
10 km	ground truth	–	–	0.00	0.00
10 km	v10 raw	6278	0	0.18	2.20
10 km	v10 + router	6282	561	0.00	0.00
25 km	ground truth	–	–	0.00	0.00
25 km	v10 raw	6278	0	0.00	0.00
25 km	v10 + router	6281	561	0.00	0.00

6.7 Qualitative examples

Figure 6 shows representative predictions on a small grid of validation routes. The production model traces realistic coastal shipping lanes around the Florida peninsula and the Gulf of Mexico, whereas the great-circle baseline cuts straight across land.

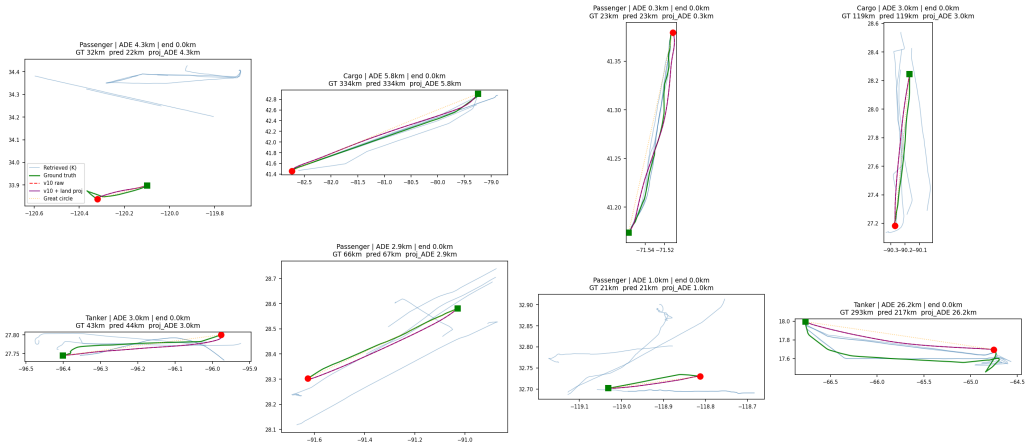
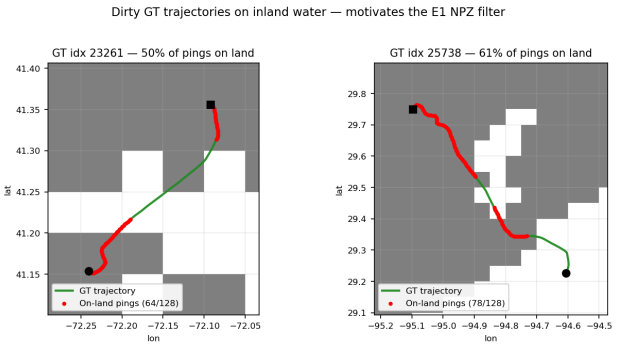


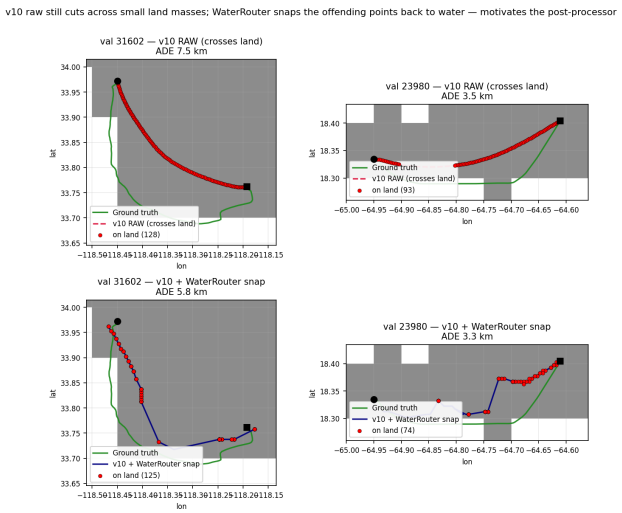
Figure 6: Qualitative examples on cleaned-corpus validation routes. Green = ground truth, red = production prediction, grey = land. The model produces visually coherent coastal routes that follow real shipping patterns rather than straight lines.

6.8 Motivating ablations

Two short failure examples illustrate why specific elements of the pipeline exist. Figure 7(a) shows two ground-truth trajectories from the unfiltered corpus that visibly track inland rivers – direct motivation for the data-level land filter. Figure 7(b) shows a raw model prediction that cuts through a peninsula on the way to its destination – direct motivation for the post-processor that snaps such points to connected water.



(a) Dirty ground truth on a river: motivates the data-level land filter.



(b) Raw model output crossing a peninsula: motivates the post-processor.

Figure 7: Two motivating failure modes that drove the principal design choices. Each panel is one concrete validation route.

6.9 Hyperparameter sensitivity

The loss weights and learning rate for v10 were initially carried forward from v5/v9 without an explicit grid search (Section 4.3). The production protocol added a small sweep around the inherited values to test whether

the configuration is on a sharp local optimum or a broad plateau. Table 10 reports the sweep, evaluated on val (5 subsample seeds) and held-out test ($n = 2326$).

Table 10: Hyperparameter sensitivity around the v10 cleaned-corpus configuration ($\lambda_{\text{end}} = 10$, $\lambda_{\text{land}} = 0.05$ selected). All other hyperparameters are held fixed (`train_seed = 42`, 60 epochs, batch 128 bf16, $\eta = 2 \times 10^{-4}$, cosine LR schedule, 5 % warmup).

Config	val ADE (raw)	val cross% (raw)	test ADE (router)	test cross%
$\lambda_{\text{land}} = 0.0$ (ablation)	6359 ± 397	3.12	8417	0.04
$\lambda_{\text{land}} = 0.01$	5925 ± 389	2.36	7764	0.04
$\boldsymbol{\lambda_{\text{land}} = 0.05}$ **(production)**	$\mathbf{5913 \pm 384}$	**1.88**	**7637**	**0.00**
$\lambda_{\text{land}} = 0.1$	5973 ± 352	2.16	7766	0.00
$\lambda_{\text{end}} = 10$ (production)	5913 ± 384	1.88	7637	0.00
$\lambda_{\text{end}} = 25$	6326 ± 404	2.60	8390	0.09

The sweep reveals three things:

1. L_{land} **has measurable effect.** Removing it ($\lambda_{\text{land}} = 0$) costs 446 m of val ADE relative to the production setting and increases the pre-snap trajectory-crossing rate from 1.88 % to 3.12 %. The snap step recovers all of the water-validity downstream, but at a higher ADE cost – so the land-aware loss meaningfully reduces the work the post-processor must do.

2. $\lambda_{\text{land}} = 0.05$ **is the HP-best setting and is therefore the production choice.** It edges out $\lambda_{\text{land}} = 0.01$ on every metric (5913 vs 5925 m val, 7637 vs 7764 m test, 1.88 vs 2.36 % pre-snap crossings, 0.00 vs 0.04 % test cross%). The val gap is within the subsample std band ($\pm$384 m) and the test gap is at the level of single-seed noise, but the rankings agree on three independent metrics (val ADE, test ADE, pre-snap cross%), so we adopt $\lambda_{\text{land}} = 0.05$ as the production setting. $\lambda_{\text{land}} = 0.1$ over-regularises slightly, confirming that the optimum is broad-and-bracketed rather than a knife-edge.

3. $\lambda_{\text{end}} = 25$ **hurts.** Raising the endpoint constraint weight from 10 to 25 costs ≈ 400 m val ADE and ≈ 750 m test ADE. The production value $\lambda_{\text{end}} = 10$ is the correct setting; the historical v2/v3 values of $\lambda_{\text{end}} = 50$ would have been actively harmful.

No grid search was attempted over the LR, warmup, batch size, λ_{smooth}, dropout, or the retrieval bank size (K, t_{retr}). The HP-sensitivity result above is therefore narrow (two knobs $\times$ small neighbourhoods), but it answers the most pressing question – L_{land} matters and a slightly larger weight than the inherited default is the right setting – and gives a plateau estimate for the production config. Source CSV: `results/tier1_eval.csv`.

6.10 Architecture-controlled ladder on the clean corpus

To isolate architectural effects from training-data quality, v4, v5 and v9 were retrained from scratch on the cleaned corpus under the same MMSI-grouped $80/15/5$ split as v10. The ranking is unchanged from the dirty-corpus ladder: v10 reduces val/test ADE relative to v9 by $\approx$ 0.95/2.0 km, while v9 in turn beats v5/v4 by only ≈ 0.4 km – so retrieval (v9$\rightarrow$v10) is the dominant single step, every other architectural perturbation is ≤ 250 m. Scheduled sampling slightly hurts on the clean corpus (v5 ≈ 200 m worse than v4 on both val and test), reversing the original single-seed dirty-corpus result. The full numbers are in Appendix B, Table 13.

6.11 Use-case ranking

Table 11 summarises which configuration is best under which deployment constraint on the cleaned corpus, the only setting in which all variants are directly comparable.

Table 11: Use-case ranking on the cleaned corpus. "Water-strict" requires cross%@10 km $= 0$. Baseline ADE numbers are in Tables 1–2.

Use case	Best configuration	ADE
Water-strict deployment	v10 + snap-only router (production)	$\mathbf{5930 \pm 372}$ val / **7637** test
Best val ADE (no water constraint)	v10 raw (tied with production)	5913 ± 384 m
Short hops only (<50 km)	Great-circle geodesic	1431 ± 54 m (short)

7 Design rationale and lessons

The production configuration described above is the last in a ladder of twelve architectural variants explored over the course of the project. Table 12 summarises which idea each variant tested and its current status; for ADE comparisons see Table 13 in Appendix B.2 (variants that re-load and run with the current model code) and Appendix E (variants that do not).

Table 12: Architectural ladder of the full-route generation variants. Parked/abandoned variants are detailed in Table 15 (Appendix E); ADE comparisons for surviving variants are in Table 13.

Variant	Idea	Status
MVP–v3	Encoder-decoder, no bearing features	superseded (unloadable)
v4	+ bearing-to-end + encoder layers	superseded
v5	v4 + scheduled sampling	superseded
v6, v7	Obstacle-conditioned generation	parked
v9	Mean-pool retrieval encoder	superseded
v10	**Per-step retrieval + land loss + water snap**	**production**
v11, v12	Halton-cone / K-ring pointer over water graph	abandoned
v13a, v13b	Length router / diffusion Transformer	planned

7.1 Per-step validation loss does not predict rollout ADE

The v2–v5 ladder improved per-step Huber loss ($0.105 \to 0.074$) without moving end-to-end ADE; the same disconnect appeared in v12 (87.2 % top-1 cell-pointer accuracy vs 27 365 m rollout ADE). Per-step metrics are useful for monitoring, but only end-to-end rollout should decide architectural choices.

7.2 Retrieval was the largest single architectural lever

Of the seven ideas tried before retrieval (bigger model, bearing features, encoder layers, scheduled sampling, obstacle conditioning, two pointer formulations), none moved ADE by more than $\approx 1\,\%$ over the MVP. Retrieval – both as a zero-training top-1 baseline and as a learned per-step bank – moved ADE by $19\,\%$ and $> 25\,\%$ respectively, and the architecture-controlled clean-corpus retraining (Table 13) confirms the ranking on matched data: every non-retrieval step is at most a ≈ 0.25 km perturbation. Retrieval is doing real work, not just adding

parameters: per-route ADE correlates with top-1 neighbour distance on the short bucket ($\rho_{\text{Spearman}} = +0.40$), collapses to zero on medium, and is weak on long ($\rho = +0.18$, $n = 174$). The practical lesson: when the training set already contains many near-neighbour examples, a non-parametric memory is worth trying before scaling the parametric model.

7.3 Data quality was the largest single intervention measured here

Holding the dirty-trained v10 weights fixed and switching evaluation to the cleaned corpus dropped water-strict ADE from 8898 ± 650 m to 6051 ± 225 m ($32\,\%$ reduction, larger than any single architectural change) and the trajectory-crossing rate from $2.00 \pm 0.67\,\%$ to $0.00 \pm 0.00\,\%$. Retraining v10 from scratch on the cleaned corpus shaves a further ≈ 120 m and restores a leak-free held-out test partition with test ADE ≈ 7637 m.

8 Summary, conclusion and outlook

8.1 Summary

This work delivered a complete maritime route generator that satisfies a strict water-validity constraint while reducing ADE by $32\,\%$ versus the great-circle baseline. Three elements drove the result: a data-quality step, a model that uses per-step retrieval to bypass the mean-pool bottleneck, and a fast post-processor that projects any residual land-violating waypoint onto connected water. The full pipeline – from raw AIS to production output – is reproducible from a single repository.

8.2 Experimental design

Each architectural change was compared against the previous best on the same val split before adoption. The headline is a five-seed mean $\pm$ std over disjoint 500-route val subsamples alongside a single-shot test number ($n = 2326$, $5\,\%$ MMSI-disjoint partition). Training-seed variance is measured across three independent runs of the production config; water-validity is reported at three thresholds (5, 10, 25 km); retrieval neighbours for val queries come from the training split only; every figure can be regenerated from a single script in the repository.

8.3 Limitations

The production model is a route generator for known waters: it interpolates patterns it has seen during training rather than reasoning about novel geographies from first principles. Five caveats matter for interpreting the headline:

Val-to-test optimism gap (Section 6.4).
The held-out test partition is $\approx 30\,\%$ harder than val for every cleaned-trained variant; val numbers are therefore optimistic by that margin and the test ADE ≈ 7637 m is the honest generalisation estimate.

Training-seed variance is small but measured.
Across three retraining runs the between-checkpoint std is 138 m on val / 179 m on test, smaller than the within-run subsample std of ≈ 360 m: the headline is robust to model-init noise, but the uncertainty band is dominated by subsample noise.

Hyperparameter search was minimal.
A narrow grid around the production config ($\lambda_{\text{land}} \in \{0, 0.01, 0.05, 0.1\}$, $\lambda_{\text{end}} \in \{10, 25\}$) places it on a plateau; LR, warmup, batch size, λ_{smooth}, dropout, and the retrieval bank size were not swept.

Long routes (>500 km).
v10 still trails the zero-training top-1 retrieval baseline on the long bucket; mitigation is the length router (future work, v13a).

FDE is not zero.
Linear endpoint correction drives the AR component to zero but the WaterRouter snap can move the final waypoint to a nearby water cell, leaving an ≈ 608 m val / 488 m test residual. FDE is kept in the tables for diagnostics, not as a primary ranking criterion.

8.4 Future work

Two follow-ups can be added without redesigning the system. **v13a, a length-conditioned router**: v10+router is ahead on every length bucket except short (< 50 km), where great-circle wins (1431 m vs 2137 m); a "great-circle below 50 km, v10+router otherwise" rule promises a small headline gain. **v13b, TrajDiff**: a whole-sequence diffusion Transformer conditioned on the same 163-token retrieval bank, with multi-scale SDF features and classifier-free guidance.

8.5 Closing remark

This report shows that data hygiene, retrieval, and a lightweight geometric post-processor together deliver a maritime route generator that is simultaneously accurate, water-valid, and fast (well under a second per trajectory on a single RTX 3090). The pattern – a learned generator paired with a structural projector onto a graph-valued constraint surface – is not AIS-specific and applies wherever the support of valid outputs is a discrete graph that the model alone cannot guarantee membership of.

All commands, configs, and trained checkpoints are documented in the repository `README.md`; the audit trail tying every reported number to its source CSV is in Appendix G.

A Training pseudocode

Algorithm 2 Training loop for the production model.

Require: training set $\mathcal{D}_{\text{train}}$, retrieval index $\mathcal{K}$, land mask M_{land}, hyper-parameters $\lambda_{\text{end}}, \lambda_{\text{smooth}}, \lambda_{\text{land}}, \theta_{\text{train}}$

Initialise G_θ, optimiser, cosine-warmup LR scheduler.
for epoch = 1 to E **do**
  for minibatch $\mathcal{B}$ in $\mathcal{D}_{\text{train}}$ **do**
    $R \leftarrow \mathcal{K}$.fetch($\mathcal{B}$) {$K = 5$ retrieved routes per query}
    $\text{mem} \leftarrow \text{Encoder}(p_1, p_T, v, R)$
    $\hat{\tau} \leftarrow \text{TeacherForcedDecoder}(\text{mem}, \tau)$
    $\mathcal{L}_\Delta \leftarrow \text{Huber}(\hat{\Delta}, \Delta)$
    $\mathcal{L}_{\text{end}} \leftarrow \|\text{cumsum}(\hat{\Delta}) + p_1 - p_T\|^2$
    $\mathcal{L}_{\text{smooth}} \leftarrow \mathbb{E}_t \|\hat{\Delta}_{t+1} - \hat{\Delta}_t\|^2$
    $\mathcal{L}_{\text{land}} \leftarrow \mathbb{E}_t \text{ReLU}\big(M_{\text{land}}.\text{sample}(\hat{p}_t) - \theta_{\text{train}}\big)^2$
    $\mathcal{L} \leftarrow \mathcal{L}_\Delta + \lambda_{\text{end}}\mathcal{L}_{\text{end}} + \lambda_{\text{smooth}}\mathcal{L}_{\text{smooth}} + \lambda_{\text{land}}\mathcal{L}_{\text{land}}$
    backprop $\mathcal{L}$, optimiser step, LR scheduler step
  end for
  evaluate ADE/FDE on validation set, persist best checkpoint
end for

B Complete results tables

This appendix gives the full quantitative ladder, split into the single-step formulation (v1, Appendix B.1) and the full-trajectory generation formulation (v2 onward, Appendix B.2). Numbers from the two formulations are not directly comparable – v1's ADE is a per-step displacement over a single one-step prediction (the natural baseline is constant-velocity extrapolation), whereas v2+'s ADE is the mean haversine error per waypoint over a generated 128-step route (the natural baseline is the great-circle geodesic).

B.1 v1 – next-step displacement prediction

Across five configurations (varying training horizon, sequence length, open-sea filter, and lighthouse-distance features) the v1 single-step Transformer matched the constant-velocity baseline on per-step ADE to within ± 1 m (raw numbers in `results/summary.csv`). This null result motivated the pivot to the full-trajectory formulation reported in the main body.

B.2 v2 onward – unified cleaned-corpus ladder

Table 13 reports every parametric and baseline variant evaluated by the unified driver (`scripts/eval/eval_all_clean.py`). All entries are the mean $\pm$ population standard deviation across the same five 500-route subsample seeds $\{0, 1, 2, 42, 123\}$ on the cleaned validation set (`trajgen_128_clean.npz`, MMSI-grouped split with `split_seed = 42`). Every number traces back to `results/headline_clean_5seed_summary.csv`.

Table 13: Unified cleaned-corpus ladder. Mean $\pm$ std over 5 disjoint 500-route subsamples; trained checkpoint held fixed.

Variant	ADE (m)	FDE (m)	land%@10 km	cross%@10 km	n_{seeds}
v4 (encoder + bearing)	7878 ± 231	0 ± 0	1.41 ± 0.26	6.00 ± 0.67	5
v5 (+ scheduled sampling)	7887 ± 225	0 ± 0	1.41 ± 0.30	6.20 ± 1.01	5
v9 (mean-pool retrieval)	7215 ± 228	0 ± 0	1.14 ± 0.26	5.40 ± 0.79	5
v10 raw	6034 ± 221	0 ± 0	0.19 ± 0.05	2.08 ± 0.45	5
v10 + hard projection	6381 ± 198	0 ± 0	0.00 ± 0.00	0.00 ± 0.00	5
v10 + router (production)	6051 ± 225	581 ± 17	0.00 ± 0.00	0.00 ± 0.00	5
Retrieval top-1 (zero training)	11667 ± 636	11832 ± 468	0.00 ± 0.00	0.00 ± 0.00	5
Great-circle baseline	8735 ± 651	0 ± 0	2.69 ± 0.44	8.80 ± 0.68	5

Pre-bearing variants (MVP, v2, v3) and abandoned variants (v6, v7, v11, v12) are not in this table; see Appendix E.

C Implementation notes

MMSI-grouped train/val split. The split keys off the *root* MMSI (stripping the batch-prefix and segment-suffix produced by the data pipeline) so every segment from one vessel lands in one partition. This closed a train/val divergence in early v1 training that was being misread as overfitting.

Cached retrieval index. Built once at corpus-preparation time over $(\text{start}_{\text{lon}}, \text{start}_{\text{lat}}, \text{end}_{\text{lon}}, \text{end}_{\text{lat}})$ in raw degrees plus a vessel-type penalty (squared-Euclidean, $vtype\_weight = 0.5$, ≈ 50 km of start-point shift per vessel-type mismatch). Validation queries retrieve only from the training split, so no val route is its own neighbour.

Checkpoint resumption. Resuming restores weights, optimiser state and scheduler position, appends to the metrics log, and strips `torch.compile` prefixes; this made the 60-epoch production run survivable across two interruptions.

Hard projection vs water-graph snap. In-loop hard projection prevents the decoder from ever seeing an on-land conditioning point but hurts ADE on the dirty corpus (the projected point is no longer the most-likely-next-step); the end-of-pipeline snap avoids that trade-off and, on the cleaned corpus, the two strategies are within seed noise – the cheaper snap is preferred (Table 13).

D Production hyperparameters

Table 14: Production hyper-parameters, verified against `runs/trajgen_v10_clean_train_lambdaland05/best.pt` and `configs/trajgen/trajgen_v10_clean_train_lambdaland05.yaml`.

Group	Parameter	Value
Architecture	d_{model}	256
	heads	8
	d_{ff}	1024
	decoder layers	4
	encoder layers (over memory)	2
	K retrieved / t_{retr}	5 / 32 (carried forward from v9, not swept)
	k_{past} (windowed mask)	32
	vessel-type vocabulary	28 codes, embed dim 8
	trainable parameters	5 842 138 ($\approx 5.84 \times 10^6$)
Loss	λ_{end}	10
	λ_{smooth}	1.0
	λ_{land}	0.05 (HP-selected; see Section 6.9)
	θ_{train} (land penalty)	10 km
	p_{ss} (sched. sampling, ep ≥ 20)	0.5
Optimisation	optimiser	AdamW ($wd = 10^{-4}$)
	peak LR / warmup / floor	2×10^{-4} / 5 % / 1 %
	batch size	128 (bf16)
	epochs (best.pt at)	60 (saved ep 52)
	gradient clipping	global norm 1.0
	dropout	0.1
Data	resampled track length T	128
	training data (cleaned)	`trajgen_128_clean.npz` (GT land%@10 km = 0)
	split fractions / split seed	80 / 15 / 5 / 42
	split grouping	MMSI-grouped (`_root_id` in `src/data.py`)
	retrieval `vtype` weight	0.5 (`build_and_cache_knn`)

E Abandoned and historical variants

Variants outside the cleaned-corpus unified comparison (Appendix B.2) are summarised in Table 15: either their checkpoints cannot reload under the current model class or the variant was abandoned for a documented structural reason. Full post-mortems are in `docs/decisions.md`.

Table 15: Abandoned and historical variants. ADE numbers are single-seed on the dataset each variant was originally trained on (*not* comparable to the unified Table 13).

Variant	Failure mode	Original ADE
MVP, v2, v3	3-D decoder input (no bearing); checkpoints unloadable under current `TrajectoryGenerator` (5-D input)	MVP ≈ 6807 m
v6 / v7	$\mathcal{L}_{obs} \equiv 0$: augmenter placed exclusion zones laterally to GT, penalty never fired	n/a (no avoidance learned)
v11	Halton-cone pointer; min cone radius ($\sim$3 km) $\gg$ median GT step ($\approx$340 m), every sample overshot	$\sim 3 \times 10^4$ m
v12	K-ring discrete pointer; 87.2 % top-1 cell accuracy coexisted with massive rollout drift (K=4 drops $\sim$10 % of GT transitions; $\lambda_{ade} < 0.01$ % of loss)	27 365 m

F Supplementary diagnostics

Training dynamics. The production v10 training/val loss plateaus around epoch 40 of 60; no early stopping was needed (the cosine schedule with 5 % warmup reaches its minimum just as the loss flattens). v9 converges to a higher val loss than v10, while the discrete-pointer v12 is markedly unstable.

Per-vessel-type breakdown. Per-class ADE on the production checkpoint (Table 16) shows short coastal hops clustering in the passenger class (median val ADE 1.6 km), with cargo and tanker dominated by longer voyages (5.1–5.6 km). The ranking is preserved on test.

Table 16: Per-class ADE (mean $\pm$ population std and median, in metres) for the production v10+router checkpoint on val (2000-route subsample, seed=42) and the full held-out test partition.

Class	n_{val}	val ADE (m)	val median (m)	n_{test}	test ADE (m)	test median (m)
passenger	883	2767 ± 5348	1574	496	3955 ± 4843	2685
cargo	686	8148 ± 10696	5113	1190	8457 ± 9815	5330
tanker	431	10072 ± 19663	5599	640	8965 ± 14910	4950

G Audit summary

The numbers in this report were produced by a single evaluation pass against the source artefacts. Production checkpoint: `runs/trajgen_v10_clean_train_lambdaland05/best.pt`, trained on `data/processed/trajgen_128_clean.npz` under an MMSI-grouped 80/15/5 split with $\lambda_{\text{land}} = 0.05$ (HP-selected, Table 10); the dirty-trained `runs/trajgen_v10/best.pt` is retained only for the data-quality

comparison. All cleaned-corpus numbers were produced by `scripts/eval/eval_all_clean.py` (unified comparison) and `eval_all_tier1.py` (architecture-controlled ladder) using the same split and the same five subsample seeds $\{0, 1, 2, 42, 123\}$, writing `results/headline_clean_5seed.csv` and `results/tier1_eval.csv` which every table and figure script reads. Pre-bearing variants (MVP, v2, v3) could not be re-evaluated under the current model class and are listed only in Appendix E.

References

[1] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George B. van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International Conference on Machine Learning (ICML)*, 2022.

[2] Samuele Capobianco, Leonardo M. Millefiori, Nicola Forti, Paolo Braca, and Peter Willett. Deep learning methods for vessel trajectory prediction based on recurrent neural networks. *IEEE Transactions on Aerospace and Electronic Systems*, 2021.

[3] Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest-neighbor language models. In *International Conference on Learning Representations (ICLR)*, 2020.

[4] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[5] Maohan Liang, Ryan Wen Liu, Shichen Li, Zhe Xiao, Xinqiang Liu, and Feng Lu. An unsupervised learning method with convolutional auto-encoder for vessel trajectory similarity computation. *Ocean Engineering*, 2022.

[6] Duong Nguyen, Rodolphe Vadaine, Guillaume Hajduch, René Garello, and Ronan Fablet. A multi-task deep learning architecture for maritime surveillance using AIS data streams. In *IEEE International Conference on Data Science and Advanced Analytics (DSAA)*, 2020.

[7] Tim Salzmann, Boris Ivanovic, Punarjay Chakravarty, and Marco Pavone. Trajectron++: Dynamically-feasible trajectory forecasting with heterogeneous data. In *European Conference on Computer Vision (ECCV)*, 2020.

[8] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[9] Quan Wang, Yang Yuan, Xiaoyong Zhu, Jie Yang, and Yi Wang. Trajectory generation with generative adversarial networks. In *ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, 2019.

[10] Paul Wessel and Walter H. F. Smith. A global, self-consistent, hierarchical, high-resolution shoreline database. *Journal of Geophysical Research*, 1996.